

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

UNIT-1

Program no.:

Write a program that asks the user for a weight in kilograms and converts it to pounds. There are 2.2 pounds in a kilogram.

Aim:

To write a Python program that takes user input for weight in kilograms and converts it to pounds using the conversion factor:

1 kilogram = 2.2 pounds

Description:.

This program:

- Prompts the user to enter a weight in kilograms.
- Converts the input to pounds using the formula:
 $\text{pounds} = \text{kilograms} \times 2.2$
- Displays the result in a readable format.

Syntax :.

```
input("Enter weight: ")
```

```
float(input(...))
```

```
pounds = kilograms * 2.2
```

```
print(...)
```

Program:

```
kilograms = float(input("Enter weight in kilograms: "))
```

```
pounds = kilograms * 2.2
```

```
print(f'{kilograms} kilograms is equal to {pounds} pounds.')
```

Output:

```
Enter weight in kilograms: 19
```

```
19.0 kilograms is equal to 41.800000000000004 pounds.
```

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

Program no.:

Write a program that uses a for loop to print the numbers 8, 11, 14, 17, 20, . . . , 83, 86, 89.

Aim.:

To write a Python program that prints a sequence of numbers starting from 8 to 89, increasing by 3 using a for loop.

Description.:

This program:

- Uses Python's range() function to generate numbers from 8 to 89.
- The range(start, stop, step) function helps control the loop:
- start = 8
- stop = 90 (since range() excludes the stop value)
- step = 3

Syntax .:

```
for i in range(...):  
    range() range(start, stop, step)  
    print() print(i)
```

Program.:

```
for i in range(8, 90, 3):  
    print( i , end=' ')
```

Output.:

8 11 14 17 20 23 26 29 32 35 38 41 44 47 50 53 56 59 62 65 68 71 74 77 80 83 86 89

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

Program no.:

Split a string into array of characters in Python.

Aim:.

To write a Python program that takes a string and splits it into a list of individual characters.

Description:.

This program:

- Accepts a string input from the user or uses a predefined string.
- Uses Python's built-in list() function to convert the string into a list of characters.
- Displays the resulting list, where each character from the original string becomes an element.

Syntax :.

```
text = "example"  
characters = list(text)  
print(characters)
```



Program:.

```
text = "hello"  
characters = list(text)  
print(characters)
```

ADITYA
UNIVERSITY

Output:.

```
['h', 'e', 'l', 'l', 'o']
```

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

Program:.

Write a Python program to get the largest number from a list.

Aim:.

To write a Python program that identifies and returns the largest number from a given list of numbers.

Description:.

This program:

- Accepts a list of numbers (either predefined or user-input).
- Uses Python's built-in max() function to find the largest number.
- Displays the result to the user.

This is useful in data analysis, ranking systems, or any scenario where you need to identify the highest value.

Syntax:.

```
numbers = [val1, val2, ...]
```

```
max(numbers)
```

```
print(...)
```

Program:.

```
numbers = [45, 12, 78, 34, 89, 23, 90, 67]
```

```
largest = max(numbers)
```

```
print(f"The largest number in the list is: {largest}")
```

Output:.

The largest number in the list is: 90

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

Program no.:

Write a Python program to calculate the nth Fibonacci number using a function.

Aim.:

To write a Python program that calculates the nth Fibonacci number using a user-defined function.

Description.:

The Fibonacci sequence is a series of numbers where:

- The first two numbers are 0 and 1.
- Each subsequent number is the sum of the previous two.

Mathematically:

- $F(0) = 0$
- $F(1) = 1$
- $F(n) = F(n-1) + F(n-2)$ for $n \geq 2$

Syntax.:

```
def function_name(parameters):  
    for i in range(...)  
    ifelse  
    int(input(...))  
    print(...)
```

Program.:

```
def fibonacci(n):  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        a, b = 0, 1  
        for _ in range(2, n + 1):  
            a, b = b, a + b
```

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

```
    return b

n = int(input("Enter the position (n) of the Fibonacci number: "))
result = fibonacci(n)
print(f"The {n}th Fibonacci number is: {result}")
```

Output:.

Enter the position (n) of the Fibonacci number: 3

The 3th Fibonacci number is: 2



A D I T Y A
U N I V E R S I T Y

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

UNIT-2

Program no.:

Write a Python program that defines a Car class with attributes like make, model, and year, and methods like start() to start the car and stop() to stop it.

Aim: To define a class Car with attributes and methods to simulate basic car operations.

Description:

This program demonstrates how to create a class with a constructor, instance attributes, and methods. It models a car with attributes like make, model, and year, and includes methods to start and stop the car.

Syntax:

class ClassName:

def __init__(self, parameters):self.attribute = value * *

def method_name(self): # method logic

Program:

class Car:

def __init__(self, make, model, year):

self.make = make

self.model = model

self.year = year

def start(self):

print(f'{self.year} {self.make} {self.model} is starting...')

def stop(self):

print(f'{self.year} {self.make} {self.model} is stopping...')

my_car = Car("Toyota", "Camry", 2022)

my_car.start()

my_car.stop()

Output:

2022 Toyota Camry is starting...

2022 Toyota Camry is stopping...

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

Program no.:

Write a Python program that demonstrates inheritance by creating a base class Animal and derived classes like Dog, Cat, etc., each with their specific behaviors.

Aim:

To demonstrate inheritance by creating a base class and derived classes with specific behaviors.

Description:

Inheritance allows one class to inherit properties and methods from another. This example shows how Dog and Cat inherit from Animal and define their own behaviors.

Syntax:

```
class BaseClass: # base methods
```

```
class DerivedClass(BaseClass): # extended or overridden methods
```

Program:

```
class Animal:
```

```
    def speak(self):
```

```
        print("Animal makes a sound")
```

```
class Dog(Animal):
```

```
    def speak(self):
```

```
        print("Dog barks")
```

```
class Cat(Animal):
```

```
    def speak(self):
```

```
        print("Cat meows")
```

```
dog = Dog()
```

```
cat = Cat()
```

```
dog.speak()
```

```
cat.speak()
```

Output:

Dog barks

Cat meows

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

Program no.:

Define a base class called Animal with a method make_sound(). Implement derived classes like Dog, Cat, and Bird that override the make_sound() method to produce different sounds. Demonstrate polymorphism by calling the method on objects of different classes.

Aim:

To demonstrate polymorphism by overriding a method in multiple derived classes.

Description:

Polymorphism allows the same method to behave differently based on the object calling it. Here, make_sound() is overridden in Dog, Cat, and Bird.

Syntax:

class Base: def method(self): pass class Derived(Base): def method(self): # override

Program:

```
class Animal:
    def make_sound(self):
        print("Some generic sound")
class Dog(Animal):
    def make_sound(self):
        print("Woof!")
class Cat(Animal):
    def make_sound(self):
        print("Meow!")
class Bird(Animal):
    def make_sound(self):
        print("Tweet!")
animals = [Dog(), Cat(), Bird()]
for animal in animals:
    animal.make_sound()
```

Output:

Woof! Meow! Tweet!

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

Program no.:

Write a Python program that demonstrates error handling using the try-except block to handle division by zero.

Aim:

To handle runtime errors using exception handling in Python.

Description:

This program uses a try-except block to catch and handle a ZeroDivisionError, preventing the program from crashing.

Syntax:

try: # risky code except
ExceptionType: # error handling

Program:

```
def divide_numbers(a, b):
```

```
    try:
```

```
        result = a / b
```

```
        print(f"The result of {a} / {b} is: {result}")
```

```
    except ZeroDivisionError:
```

```
        print("Error: Division by zero is not allowed.")
```

```
if __name__ == "__main__":
```

```
    divide_numbers(21, 2)
```

```
    divide_numbers(8, 0)
```

Output:

The result of 21 / 2 is: 10.5

Error: Division by zero is not allowed.

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

UNIT-3

Program no.:

To get help on the add function give aim,discription,syntax, python program

Aim:

To create a function that takes two numbers as input and returns their sum.

Description:

The add() function is a user-defined function that performs addition of two numeric values. It accepts two parameters, computes their sum, and returns the result. This is useful for modularizing code and reusing the addition logic wherever needed.

Syntax:

```
def add(a, b):  
    return a + b
```

Program:

```
def add(a, b):  
    return a + b
```

```
num1 = int(input("Enter first number: "))  
num2 = int(input("Enter second number: "))
```

```
result = add(num1, num2)  
print("The sum is:", result)
```

Output:

Enter first number: 12

Enter second number: 8

The sum is: 20



ADITYA
UNIVERSITY

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

Program no.:

To test whether none of the elements of a given array is zero.

Aim:

To check whether all elements in a given array are non-zero.

Description:

This function iterates through each element of the array and verifies that none of them is equal to zero. If any element is zero, the function returns False; otherwise, it returns True. This is useful for validating input arrays before performing operations like division, logarithms, or matrix inversion.

Syntax:

```
def no_zero_elements(arr):
```

```
    for element in arr:
```

```
        if element == 0:
```

```
            return False    return True
```

Program:

```
def no_zero_elements(arr):
```

```
    for element in arr:
```

```
        if element == 0:
```

```
            return False
```

```
    return True
```

```
array = list(map(int, input("Enter array elements separated by space: ").split()))
```

```
if no_zero_elements(array):
```

```
    print("☑ All elements are non-zero.")
```

```
else:
```

```
    print("✗ At least one element is zero.")
```

Output-1:

Enter array elements separated by space: 4 0 6 9

✗ At least one element is zero.

Output-2:

Enter array elements separated by space: 3 5 7 2

☑ All elements are non-zero.



A D I T Y A
U N I V E R S I T Y

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

Program no.:

To create an element-wise comparison (greater, greater_equal, less and less_equal, equal, equal within a tolerance) of two given arrays.

Aim:

To perform element-wise comparison operations (greater, greater_equal, less, less_equal, equal, and equal within a tolerance) between two NumPy arrays.

Description:

Element-wise comparison is used to compare corresponding elements of two arrays. NumPy provides built-in functions that return Boolean arrays indicating the result of each comparison. These are useful in filtering, masking, and conditional logic.

Supported comparison functions:

- `np.greater(a, b) → True if  $a[i] > b[i]$`
- `np.greater_equal(a, b) → True if  $a[i] \geq b[i]$`
- `np.less(a, b) → True if  $a[i] < b[i]$`
- `np.less_equal(a, b) → True if  $a[i] \leq b[i]$`
- `np.equal(a, b) → True if  $a[i] == b[i]$`
- `np.allclose(a, b, atol=...) → True if  $|a[i] - b[i]| \leq \text{atol}$`

Syntax:

```
import numpy as np
np.greater(array1, array2)
np.greater_equal(array1, array2)
np.less(array1, array2)
np.less_equal(array1, array2)
np.equal(array1, array2)
np.allclose(array1, array2, atol=value)
```

Program:

```
import numpy as np
array1 = np.array([10, 20, 30, 40, 50])
array2 = np.array([15, 20, 25, 40, 60])
```

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

```
print("Array 1:", array1)
print("Array 2:", array2)
print("\nElement-wise Comparisons:")
print("Greater:      ", np.greater(array1, array2))
print("Greater Equal: ", np.greater_equal(array1, array2))
print("Less:          ", np.less(array1, array2))
print("Less Equal:    ", np.less_equal(array1, array2))
print("Equal:         ", np.equal(array1, array2))
tolerance = 5
print("\nEqual within tolerance ( $\pm$ {}):".format(tolerance), np.allclose(array1, array2, atol=tolerance))
```

Output:

Array 1: [10 20 30 40 50]

Array 2: [15 20 25 40 60]

Element-wise Comparisons:

Greater: [False False True False False]

Greater Equal: [False True True True False]

Less: [True False False False True]

Less Equal: [True True False True True]

Equal: [False True False True False]

Equal within tolerance (± 5): False



Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

Program no.:

Write a NumPy program using NumPy methods - max, min, argmax, argmin, argmax, repr, count, bincount, unique.

Aim:

To demonstrate the use of NumPy methods: max, min, argmax, argmin, repr, count_nonzero, bincount, and unique on a given array.

Description:

NumPy provides powerful functions to analyze and manipulate arrays:

- np.max() / np.min() → find maximum and minimum values
- np.argmax() / np.argmin() → find indices of max and min values
- repr() → get string representation of an array
- np.count_nonzero() → count non-zero elements
- np.bincount() → count frequency of integers (non-negative only)
- np.unique() → extract unique elements from an array

Syntax:

np.max(array)

np.min(array)

np.argmax(array)

np.argmin(array)

repr(array)

np.count_nonzero(array)

np.bincount(array)

np.unique(array)

Program:

```
import numpy as np
arr = np.array([1, 2, 2, 3, 4, 4, 4, 5])
print("Original array:", arr)
print("\nUsing repr():")
print(repr(arr))
print("\nUsing count():")
```

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

```
arr_list = arr.tolist()
print("Count of 4 in array:", arr_list.count(4))
print("\nUsing np.bincount():")
print("Element counts (by index):", np.bincount(arr))
print("\nUsing np.unique():")
unique_elements, counts = np.unique(arr, return_counts=True)
print("Unique elements:", unique_elements)
print("Counts:", counts)
```

Output:

Original array: [1 2 2 3 4 4 4 5]

Using repr():

array([1, 2, 2, 3, 4, 4, 4, 5])

Using count():

Count of 4 in array: 3

Using np.bincount():

Element counts (by index): [0 1 2 1 3 1]

Using np.unique():

Unique elements: [1 2 3 4 5]

Counts: [1 2 1 3 1]



A D I T Y A
U N I V E R S I T Y

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

Program no.:

To extract all numbers from a given array which are less and greater than a specified number.

Aim

To extract all elements from a NumPy array that are either less than or greater than a specified number using comparison and filtering techniques.

Syntax:

```
import numpy as np  
array[array < value]  
array[array > value]
```

Program:

```
import numpy as np  
arr = np.array([5, 12, 7, 20, 3, 15, 8])  
threshold = 10  
less_than = arr[arr < threshold]  
greater_than = arr[arr > threshold]  
print("Original Array:", arr)  
print("Values less than", threshold, ":", less_than)  
print("Values greater than", threshold, ":", greater_than)
```



Output:

```
Original Array: [ 5 12  7 20  3 15  8]  
Values less than 10 : [5 7 3 8]  
Values greater than 10 : [12 20 15]
```

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

Program no.:

To find the indices of the maximum and minimum numbers along the given axis of an array. give each question answer like aim,discription,syntax ,python program

Aim

To find the indices of the maximum and minimum values along a specified axis in a NumPy array using `argmax()` and `argmin()`.

Description

NumPy provides:

- `np.argmax(array, axis)` → returns index of max value along axis
- `np.argmin(array, axis)` → returns index of min value along axis

This is useful for locating extreme values in rows or columns of matrices.

Syntax:

`np.argmax(array, axis=0)` # column-wise

`np.argmin(array, axis=1)` # row-wise

Program:

```
import numpy as np
```

```
arr = np.array([[10, 20, 5],  
                [15, 3, 25],  
                [30, 8, 12]])
```

```
print("Original array:")
```

```
print(arr)
```

```
max_indices = np.argmax(arr, axis=1)
```

```
min_indices = np.argmin(arr, axis=1)
```

```
print("\nIndices of the maximum values along each row:", max_indices)
```

```
print("Indices of the minimum values along each row:", min_indices)
```

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

```
max_values = arr[np.arange(len(arr)), max_indices]
min_values = arr[np.arange(len(arr)), min_indices]
print("\nMaximum values:", max_values)
print("Minimum values:", min_values)
```

Output:

Original array:

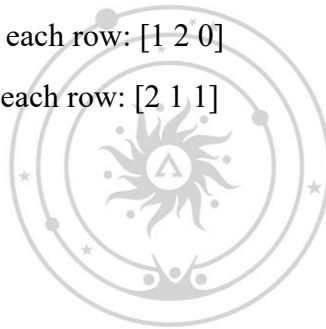
```
[[10 20 5]
 [15 3 25]
 [30 8 12]]
```

Indices of the maximum values along each row: [1 2 0]

Indices of the minimum values along each row: [2 1 1]

Maximum values: [20 25 30]

Minimum values: [5 3 8]



ADITYA
UNIVERSITY

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

UNIT-4

Program no.:

Write a Pandas program to create and display a one-dimensional array-like object containing an array of data using Pandas module.

Aim

To write a Python program using the Pandas module to create and display a one-dimensional array-like object (Series) containing an array of data.

Description

Pandas provides a powerful data structure called Series, which is a one-dimensional labeled array capable of holding any data type. This program demonstrates how to create a Series from a list and display its contents.

Syntax

`pandas.Series(data, index, dtype)`

Program:

```
import pandas as pd
data = [10, 20, 30, 40, 50]
series = pd.Series(data)
print("Pandas Series:")
print(series)
```

Output:

Pandas Series:

0 10

1 20

2 30

3 40

4 50

dtype: int64



A D I T Y A
U N I V E R S I T Y

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

Program no.:

Write a Pandas program to convert a Panda module Series to Python list and it's type.

Aim

To write a Python program using the Pandas module to convert a Pandas Series into a native Python list and display its type.

Syntax

`series.tolist()    type(series.tolist())`

- `tolist()` converts the Series to a Python list.
- `type()` returns the data type of the converted object.

Program:

```
import pandas as pd
data = pd.Series([10, 20, 30, 40, 50])
print("Pandas Series:")
print(data)
py_list = data.tolist()
print("\nConverted to Python list:")
print(py_list)
print("\nType of the converted object:",type(py_list))
```



A D I T Y A
U N I V E R S I T Y

Output:

Pandas Series:

```
0    10
1    20
2    30
3    40
4    50
```

dtype: int64

Converted to Python list:

```
[10, 20, 30, 40, 50]
```

Type of the converted object: <class 'list'>

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

Program no.:

Write a Pandas program to create and display a DataFrame from a specified dictionary data which has the index labels.

Aim

To write a Python program using the Pandas module to create and display a DataFrame from a specified dictionary, including custom index labels.

Description

A DataFrame is a two-dimensional labeled data structure in Pandas, similar to a table in a database or an Excel spreadsheet. This program demonstrates how to construct a DataFrame using a dictionary where keys represent column names and values are lists of column data. Index labels are explicitly defined to customize row identifiers.

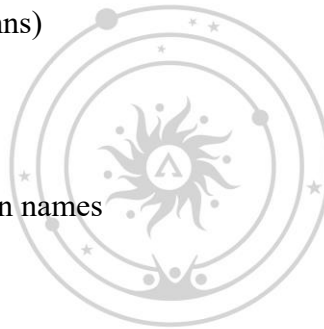
Syntax

`pandas.DataFrame(data, index, columns)`

data: dictionary of column data

index: list of row labels

columns: optional list of column names



Program:

```
import pandas as pd
import numpy as np
exam_data = {
    'name': ['Anastasia', 'Dima', 'Katherine', 'James', 'Emily',
            'Michael', 'Matthew', 'Laura', 'Kevin', 'Jonas'],
    'score': [12.5, 9, 16.5, np.nan, 9, 20, 14.5, np.nan, 8, 19],
    'attempts': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],
    'qualify': ['yes', 'no', 'yes', 'no', 'no',
               'yes', 'yes', 'no', 'no', 'yes']
}

labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']
df = pd.DataFrame(exam_data, index=labels)
```

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

```
print("Pandas DataFrame with index labels:")  
print(df)
```

Output:

Pandas DataFrame with index labels:

	name	score	attempts	qualify
a	Anastasia	12.5	1	yes
b	Dima	9.0	3	no
c	Katherine	16.5	2	yes
d	James	NaN	3	no
e	Emily	9.0	2	no
f	Michael	20.0	3	yes
g	Matthew	14.5	1	yes
h	Laura	NaN	1	no
i	Kevin	8.0	2	no
j	Jonas	19.0	1	yes



A D I T Y A
U N I V E R S I T Y

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

Program no.:

Write a Pandas program to change the name 'James' to 'Suresh' in name column of the DataFrame.

Aim

To write a Python program using the Pandas module to change the name 'James' to 'Suresh' in the Name column of a DataFrame.

Description

Pandas allows easy manipulation of tabular data using DataFrames. This program demonstrates how to locate and replace a specific value ('James') in the Name column with a new value ('Suresh') using the `replace()` method.

Syntax

`DataFrame['column_name'].replace(old_value, new_value, inplace=True)`

- 'column_name': the target column
- old_value: value to be replaced
- new_value: replacement value
- inplace=True: modifies the DataFrame directly

Program:

```
import pandas as pd
import numpy as np

exam_data = {
    'name': ['Anastasia', 'Dima', 'Katherine', 'James', 'Emily',
            'Michael', 'Matthew', 'Laura', 'Kevin', 'Jonas'],
    'score': [12.5, 9, 16.5, np.nan, 9, 20, 14.5, np.nan, 8, 19],
    'attempts': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],
    'qualify': ['yes', 'no', 'yes', 'no', 'no',
              'yes', 'yes', 'no', 'no', 'yes']
}

labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']
df = pd.DataFrame(exam_data, index=labels)
print("Original DataFrame:")
print(df)
```

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

```
df.loc[df['name'] == 'James', 'name'] = 'Suresh'  
print("\nDataFrame after changing 'James' to 'Suresh':")  
print(df)
```

Output:

Original DataFrame:

	name	score	attempts	qualify
a	Anastasia	12.5	1	yes
b	Dima	9.0	3	no
c	Katherine	16.5	2	yes
d	James	NaN	3	no
e	Emily	9.0	2	no
f	Michael	20.0	3	yes
g	Matthew	14.5	1	yes
h	Laura	NaN	1	no
i	Kevin	8.0	2	no
j	Jonas	19.0	1	yes



ADITYA
UNIVERSITY

DataFrame after changing 'James' to 'Suresh':

	name	score	attempts	qualify
a	Anastasia	12.5	1	yes
b	Dima	9.0	3	no
c	Katherine	16.5	2	yes
d	Suresh	NaN	3	no
e	Emily	9.0	2	no
f	Michael	20.0	3	yes
g	Matthew	14.5	1	yes
h	Laura	NaN	1	no
i	Kevin	8.0	2	no
j	Jonas	19.0	1	yes

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

Program no.:

Write a Pandas program to insert a new column in existing DataFrame.

Aim

To write a Pandas program to insert a new column into an existing DataFrame.

Description

Pandas allows dynamic modification of DataFrames. A new column can be added by assigning a list or Series to a new column label.

Syntax

```
DataFrame['new_column_name'] = new_data
```

Program:

```
import pandas as pd
```

```
import numpy as np
```

```
exam_data = {  
    'name': ['Anastasia', 'Dima', 'Katherine', 'James', 'Emily', 'Michael',  
            'Matthew', 'Laura', 'Kevin', 'Jonas'],  
    'score': [12.5, 9, 16.5, np.nan, 9, 20, 14.5, np.nan, 8, 19],  
    'attempts': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],  
    'qualify': ['yes', 'no', 'yes', 'no', 'no', 'yes', 'yes', 'no', 'no', 'yes']  
}
```

```
labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']
```

```
df = pd.DataFrame(exam_data, index=labels)
```

```
df['age'] = [21, 23, 22, 20, 21, 24, 23, 22, 25, 20]
```

```
print(df)
```

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

Output:

	name	score	attempts	qualify	age
a	Anastasia	12.5	1	yes	21
b	Dima	9.0	3	no	23
c	Katherine	16.5	2	yes	22
d	James	NaN	3	no	20
e	Emily	9.0	2	no	21
f	Michael	20.0	3	yes	24
g	Matthew	14.5	1	yes	23
h	Laura	NaN	1	no	22
i	Kevin	8.0	2	no	25
j	Jonas	19.0	1	yes	20



A D I T Y A
UNIVERSITY

Date:

Roll No.:

2	4	B	1	1	A	I	0	9	3
---	---	---	---	---	---	---	---	---	---

Program no.:

Write a Pandas program to get list from DataFrame column headers.

Aim:

To write a Python program using the Pandas module to extract column headers from a DataFrame as a Python list.

Description:

In Pandas, the column headers of a DataFrame can be accessed using the .columns attribute. Converting this to a list allows easy manipulation or export of column names for further processing.

Syntax:

`list(DataFrame.columns) # or DataFrame.columns.to_list()`

Program:

```
import pandas as pd
import numpy as np
exam_data = {
    'name': ['Anastasia', 'Dima', 'Katherine', 'James', 'Emily', 'Michael',
            'Matthew', 'Laura', 'Kevin', 'Jonas'],
    'score': [12.5, 9, 16.5, np.nan, 9, 20, 14.5, np.nan, 8, 19],
    'attempts': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],
    'qualify': ['yes', 'no', 'yes', 'no', 'no', 'yes', 'yes', 'no', 'no', 'yes']
}
labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']
df = pd.DataFrame(exam_data, index=labels)
col_list = df.columns.tolist()
print("List of column headers:")
print(col_list)
```

Output:

List of column headers:

`['name', 'score', 'attempts', 'qualify']`